NAME : DEVNGI BORAD (24BAM032)

SUBJECT : DIGITAL ELECTRONICS (2CS507CC23)

PRACTICAL NO.: 6

AIM: Demonstrate the working of basic logic gates and verify it with Verilog HDL.

AND GATE

CODE:

```
module and_gate(
  input A,
  input B,
  output Y
);
assign Y = A & B;
endmodule
module and_gate_tb;
  reg tb_A;
  reg tb_B;
  wire tb_Y;
and_gate dut(
  .A(tb_A),
  .B(tb_B),
  .Y(tb_Y)
);
 initial begin
   tb_A = 1'b0; tb_B = 1'b0; #10; // A=0, B=0, wait 10 time units
   tb_A = 1'b0; tb_B = 1'b1; #10; // A=0, B=1, wait 10 time units
   tb_A = 1'b1; tb_B = 1'b0; #10; // A=1, B=0, wait 10 time units
   tb_A = 1'b1; tb_B = 1'b1; #10; // A=1, B=1, wait 10 time units
     $finish ;
```

```
  end
 initial
  begin
  $monitor("Time=%0t: A=%b, B=%b, Y=%b", $time, tb_A, tb_B, tb_Y);
  end
endmodule
```

OUTPUT:

```
Time=0: A=0, B=0, Y=0
Time=10: A=0, B=1, Y=0
Time=20: A=1, B=0, Y=0
Time=30: A=1, B=1, Y=1
```

OR GATE

CODE:

```
module or_gate(
  input A,
  input B,
  output Y
);
assign Y = A || B;
endmodule
module or_gate_tb;
  reg tb_A;
  reg tb_B;
  wire tb_Y;
or_gate dut(
```

```
  .A(tb_A),
  .B(tb_B),
  .Y(tb_Y)
);
 initial begin
   tb_A = 1'b0; tb_B = 1'b0; #10; // A=0, B=0, wait 10 time units
   tb_A = 1'b0; tb_B = 1'b1; #10; // A=0, B=1, wait 10 time units
   tb_A = 1'b1; tb_B = 1'b0; #10; // A=1, B=0, wait 10 time units
   tb_A = 1'b1; tb_B = 1'b1; #10; // A=1, B=1, wait 10 time units
     $finish ;
   end
 initial
   begin
   $monitor("Time=%0t: A=%b, B=%b, Y=%b", $time, tb_A, tb_B, tb_Y);
   end
endmodule
```

OUTPUT:

```
Time=0: A=0, B=0, Y=0
Time=10: A=0, B=1, Y=1
Time=20: A=1, B=0, Y=1
Time=30: A=1, B=1, Y=1
```

NAND GATE

CODE:

```
module nand_gate(
  input A,
  input B,
  output Y
);
assign Y = ~(A & B);
endmodule
module nand_gate_tb;
  reg tb_A;
  reg tb_B;
  wire tb_Y;
nand_gate dut(
  .A(tb_A),
  .B(tb_B),
  .Y(tb_Y)
);
 initial begin
   tb_A = 1'b0; tb_B = 1'b0; #10; // A=0, B=0, wait 10 time units
   tb_A = 1'b0; tb_B = 1'b1; #10; // A=0, B=1, wait 10 time units
   tb_A = 1'b1; tb_B = 1'b0; #10; // A=1, B=0, wait 10 time units
   tb_A = 1'b1; tb_B = 1'b1; #10; // A=1, B=1, wait 10 time units
     $finish ;
   end
 initial
   begin
```

```
  $monitor("Time=%0t: A=%b, B=%b, Y=%b", $time, tb_A, tb_B, tb_Y);
  end
endmodule
```

OUTPUT:

```
Time=0: A=0, B=0, Y=1
Time=10: A=0, B=1, Y=1
Time=20: A=1, B=0, Y=1
Time=30: A=1, B=1, Y=0
```

NOR GATE

CODE:

```
module nand_gate(
  input A,
  input B,
  output Y
);
assign Y = ~(A & B);
endmodule
module nand_gate_tb;
  reg tb_A;
  reg tb_B;
  wire tb_Y;
nand_gate dut(
  .A(tb_A),
  .B(tb_B),
  .Y(tb_Y)
```

```
);
  initial begin
    tb_A = 1'b0; tb_B = 1'b0; #10; // A=0, B=0, wait 10 time units
    tb_A = 1'b0; tb_B = 1'b1; #10; // A=0, B=1, wait 10 time units
    tb_A = 1'b1; tb_B = 1'b0; #10; // A=1, B=0, wait 10 time units
    tb_A = 1'b1; tb_B = 1'b1; #10; // A=1, B=1, wait 10 time units
      $finish ;
    end
  initial
    begin
    $monitor("Time=%0t: A=%b, B=%b, Y=%b", $time, tb_A, tb_B, tb_Y);
    end
endmodule
```

OUTPUT:

```
Time=0: A=0, B=0, Y=1
Time=10: A=0, B=1, Y=0
Time=20: A=1, B=0, Y=0
Time=30: A=1, B=1, Y=0
```

X-OR GATE

CODE:

```
module nand_gate(
   input A,
   input B,
   output Y);
assign Y = ~(A & B);
```

```
endmodule
module nand_gate_tb;
  reg tb_A;
  reg tb_B;
  wire tb_Y;
nand_gate dut(
  .A(tb_A),
  .B(tb_B),
  .Y(tb_Y)
);
 initial begin
   tb_A = 1'b0; tb_B = 1'b0; #10; // A=0, B=0, wait 10 time units
   tb_A = 1'b0; tb_B = 1'b1; #10; // A=0, B=1, wait 10 time units
   tb_A = 1'b1; tb_B = 1'b0; #10; // A=1, B=0, wait 10 time units
   tb_A = 1'b1; tb_B = 1'b1; #10; // A=1, B=1, wait 10 time units
     $finish ;
   end
 initial
   begin
   $monitor("Time=%0t: A=%b, B=%b, Y=%b", $time, tb_A, tb_B, tb_Y);
   end
endmodule
```

OUTPUT:

```
Time=0: A=0, B=0, Y=0
Time=10: A=0, B=1, Y=1
Time=20: A=1, B=0, Y=1
Time=30: A=1, B=1, Y=0
```

X-NOR GATE

CODE:

```
module nand_gate(
  input A,
  input B,
  output Y
);
assign Y = ~(A & B);
endmodule
module nand_gate_tb;
  reg tb_A;
  reg tb_B;
  wire tb_Y;
nand_gate dut(
  .A(tb_A),
  .B(tb_B),
  .Y(tb_Y)
);
 initial begin
   tb_A = 1'b0; tb_B = 1'b0; #10; // A=0, B=0, wait 10 time units
   tb_A = 1'b0; tb_B = 1'b1; #10; // A=0, B=1, wait 10 time units
   tb_A = 1'b1; tb_B = 1'b0; #10; // A=1, B=0, wait 10 time units
   tb_A = 1'b1; tb_B = 1'b1; #10; // A=1, B=1, wait 10 time units
     $finish ;
   end
 initial
   begin
```

```
    $monitor("Time=%0t: A=%b, B=%b, Y=%b", $time, tb_A, tb_B, tb_Y);
    end
endmodule
```

OUTPUT:

```
Time=0: A=0, B=0, Y=1
Time=10: A=0, B=1, Y=0
Time=20: A=1, B=0, Y=0
Time=30: A=1, B=1, Y=1
```

NOT GATE

CODE:

```
module not_gate(
   input A,
   output Y
);
assign Y = ~(A);
endmodule
module not_gate_tb;
   reg tb_A;
   wire tb_Y;
not_gate dut(
   .A(tb_A),
   .Y(tb_Y)
);
  initial begin
    tb_A = 1'b0; #10; // A=0,wait 10 time units
```

```
  tb_A = 1'b1; #10; // A=1,wait 10 time units
    $finish ;
  end
 initial begin
  $monitor("Time=%0t: A=%b, Y=%b", $time, tb_A, tb_Y);
  end
endmodule
```

OUTPUT :

```
Time=0: A=0, Y=1
Time=10: A=1, Y=0
```

NAME : DEVNGI BORAD (24BAM032)

SUBJECT : DIGITAL ELECTRONICS (2CS507CC23)

PRACTICAL NO.: 6

AIM: Design and Implementation of Half and Full Adder and Subtractor circuits.

HALF ADDER

CODE:

```
module half_adder(
   input A,
   input B,
   output Y,
   output carry
);
assign Y = A ^ B;
assign carry = A & B;
endmodule
module half_adder_tb;
reg tb_A;
reg tb_B;
wire tb_Y;
wire tb_carry;
half_adder dut(
.A(tb_A),
.B(tb_B),
.Y(tb_Y),
.carry(tb_carry)
);
initial begin
tb_A = 1'b0; tb_B = 1'b0; #10;
```

```
tb_A = 1'b0; tb_B = 1'b1; #10;

tb_A = 1'b1; tb_B = 1'b0; #10;

tb_A = 1'b1; tb_B = 1'b1; #10;

$finish ;

end

initial

begin

$monitor("Time=%0t: A=%b, B=%b, Y=%b, carry=%b", $time, tb_A, tb_B, tb_Y, tb_carry);

end

endmodule
```

OUTPUT :

```
Time=0: A=0, B=0, Y=0, carry=0
Time=10: A=0, B=1, Y=1, carry=0
Time=20: A=1, B=0, Y=1, carry=0
Time=30: A=1, B=1, Y=0, carry=1
```

FULL ADDER

CODE:

```
module full_adder(
   input A,
   input B,
   input C,
   output Y,
   output carry
);

assign Y = A ^ B ^ C;
assign carry = (A & B) || (A ^ B) & C;

endmodule

module full_adder_tb;
reg tb_A;
reg tb_B;
reg tb_C;
wire tb_Y;
wire tb_carry;

full_adder dut(
```

```
.A(tb_A),
.B(tb_B),
.C(tb_C),
.Y(tb_Y),
.carry(tb_carry)
);

initial begin
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b1; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b1; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b1; #10;
tb_A = 1'b1; tb_B = 1'b1; tb_C = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b1; tb_C = 1'b1; #10;

$finish ;
end
initial
begin
$monitor("Time=%0t: A=%b, B=%b, C=%b, Y=%b, carry=%b", $time, tb_A, tb_B, tb_C,
tb_Y, tb_carry);
end
endmodule
```

OUTPUT :

```
Time=0: A=0, B=0, C=0, Y=0, carry=0
Time=10: A=0, B=0, C=1, Y=1, carry=0
Time=20: A=0, B=1, C=0, Y=1, carry=0
Time=30: A=0, B=1, C=1, Y=0, carry=1
Time=40: A=1, B=0, C=0, Y=1, carry=0
Time=50: A=1, B=0, C=1, Y=0, carry=1
Time=60: A=1, B=1, C=0, Y=0, carry=1
Time=70: A=1, B=1, C=1, Y=1, carry=1
```

HALF ADDER

CODE :

```
module half_sub(

   input A,

   input B,

   output Y,
```

```
    output borrow
);
assign Y = A ^ B;
assign borrow = ~A & B;
endmodule
module half_sub_tb;
reg tb_A;
reg tb_B;
wire tb_Y;
wire tb_borrow;
half_sub dut(
.A(tb_A),
.B(tb_B),
.Y(tb_Y),
.borrow(tb_borrow)
);
initial begin
tb_A = 1'b0; tb_B = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b1; #10;
tb_A = 1'b1; tb_B = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b1; #10;
$finish ;
end
initial
begin
$monitor("Time=%0t: A=%b, B=%b, Y=%b, borrow=%b", $time, tb_A, tb_B, tb_Y,
tb_borrow);
End
```

```
endmodule
```

OUTPUT :

```
Time=0: A=0, B=0, Y=0, borrow=0
Time=10: A=0, B=1, Y=1, borrow=1
Time=20: A=1, B=0, Y=1, borrow=0
Time=30: A=1, B=1, Y=0, borrow=0
```

FULL SUBSTRACTOR

CODE :

```
module full_sub(
   input A,
   input B,
   input C,
   output Y,
   output borrow
);
assign Y = A ^ B ^ C;
assign borrow = (~A & B) || (~A ^ B) & C;
endmodule
module full_sub_tb;
reg tb_A;
reg tb_B;
reg tb_C;
wire tb_Y;
wire tb_borrow;
full_sub dut(
.A(tb_A),
.B(tb_B),
.C(tb_C),
```

```
.Y(tb_Y),
.borrow(tb_borrow)
);
initial begin
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b1; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b1; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b1; #10;
tb_A = 1'b1; tb_B = 1'b1; tb_C = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b1; tb_C = 1'b1; #10;
$finish ;
end
initial
begin
$monitor("Time=%0t: A=%b, B=%b, C=%b, Y=%b, borrow=%b", $time, tb_A, tb_B, tb_C, tb_Y, tb_borrow);
end
endmodule
```

OUTPUT :

```
Time=0: A=0, B=0, C=0, Y=0, borrow=0
Time=10: A=0, B=0, C=1, Y=1, borrow=1
Time=20: A=0, B=1, C=0, Y=1, borrow=1
Time=30: A=0, B=1, C=1, Y=0, borrow=1
Time=40: A=1, B=0, C=0, Y=1, borrow=0
Time=50: A=1, B=0, C=1, Y=0, borrow=0
Time=60: A=1, B=1, C=0, Y=0, borrow=0
Time=70: A=1, B=1, C=1, Y=1, borrow=1
```

NAME : DEVNGI BORAD (24BAM032)

SUBJECT : DIGITAL ELECTRONICS (2CS507CC23)

PRACTICAL NO.: 6

AIM: Design and Implementation of a combination circuit that converts the given BCD number into an equivalent 2421 number.

CODE:

```
module BCD(
    input A,
    input B,
    input C,
    input D,
    output W,
    output X,
    output Y,
    output Z
);

assign W = (B & D) || (B & C) || A;
assign X = (B & ~D) || (B & C) || A;
assign Y = (~B & C) || (B & ~C & D) || A ;
assign Z = D ;

endmodule

module BCD_tb;
reg tb_A;
reg tb_B;
reg tb_C;
```

```
reg tb_D;
wire tb_W;
wire tb_X;
wire tb_Y;
wire tb_Z;


BCD dut(
.A(tb_A),
.B(tb_B),
.C(tb_C),
.D(tb_D),
.W(tb_W),
.X(tb_X),
.Y(tb_Y),
.Z(tb_Z)
);

initial begin
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b0; tb_D = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b0; tb_D = 1'b1; #10;
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b1; tb_D = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b1; tb_D = 1'b1; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b0; tb_D = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b0; tb_D = 1'b1; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b1; tb_D = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b1; tb_D = 1'b1; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b0; tb_D = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b0; tb_D = 1'b1; #10;
```

```
$finish ;
end
initial
begin
$display("   BCD NUMBER       TO     2 4 2 1 NUMBER ");
$display("-------------------------------------");
$display("A   B   C   D   ||    W   X   Y   Z    ");
$display("-------------------------------------");
$monitor("%b   %b   %b   %b   ||    %b   %b   %b   %b    ", tb_A, tb_B, tb_C, tb_D, tb_W, tb_X, tb_Y, tb_Z);
end
endmodule
```

OUTPUT :

```
BCD NUMBER       TO     2 4 2 1 NUMBER
-------------------------------------
A   B   C   D   ||    W   X   Y   Z
-------------------------------------
0   0   0   0   ||    0   0   0   0
0   0   0   1   ||    0   0   0   1
0   0   1   0   ||    0   0   1   0
0   0   1   1   ||    0   0   1   1
0   1   0   0   ||    0   1   0   0
0   1   0   1   ||    1   0   1   1
0   1   1   0   ||    1   1   0   0
0   1   1   1   ||    1   1   0   1
1   0   0   0   ||    1   1   1   0
1   0   0   1   ||    1   1   1   1
```

AIM: Design and Implementation of a combination circuit that converts a decimal digit from 84-2-1 code to 5421 code.

CODE:

```
module decimal(
   input A,
   input B,
   input C,
   input D,
   output W,
   output X,
   output Y,
   output Z
);

assign W = (A & ~C & ~D) || (A & B);
assign X = (B & ~C & ~D) || (A & D) || (A & C);
assign Y = (C ^ D) || (A & ~C);
assign Z = (A & ~C) || (~B & D) || (~A & D);

endmodule

module decimal_tb;
reg tb_A;
reg tb_B;
reg tb_C;
reg tb_D;
wire tb_W;
wire tb_X;
wire tb_Y;
wire tb_Z;


decimal dut(
.A(tb_A),
.B(tb_B),
.C(tb_C),
.D(tb_D),
.W(tb_W),
.X(tb_X),
.Y(tb_Y),
.Z(tb_Z)
);

initial begin
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b0; tb_D = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b1; tb_D = 1'b1; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b1; tb_D = 1'b0; #10;
```

```
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b0; tb_D = 1'b1; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b0; tb_D = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b1; tb_D = 1'b1; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b1; tb_D = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b0; tb_D = 1'b1; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b0; tb_D = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b1; tb_C = 1'b1; tb_D = 1'b1; #10;

$finish ;
end
initial
begin
$display("   8 4 -2 -1 NUMBER  TO     5 4 2 1 NUMBER ");
$display("-------------------------------------");
$display("A   B   C   D    ||    W   X   Y   Z    ");
$display("-------------------------------------");
$monitor("%b   %b   %b   %b    ||    %b   %b   %b   %b    ", tb_A, tb_B, tb_C, tb_D, tb_W,
tb_X, tb_Y, tb_Z);
end
endmodule
```

OUTPUT :

```
8 4 -2 -1 NUMBER  TO     5 4 2 1 NUMBER
-------------------------------------
A   B   C   D    ||    W   X   Y   Z
-------------------------------------
0   0   0   0    ||    0   0   0   0
0   1   1   1    ||    0   0   0   1
0   1   1   0    ||    0   0   1   0
0   1   0   1    ||    0   0   1   1
0   1   0   0    ||    0   1   0   0
1   0   1   1    ||    0   1   0   1
1   0   1   0    ||    0   1   1   0
1   0   0   1    ||    0   1   1   1
1   0   0   0    ||    1   0   1   1
1   1   1   1    ||    1   1   0   0
```

AIM: Design and implementation of a combinational circuit that generates the square of a given 3 bit number.

CODE :

```
module square(
   input X,
   input Y,
   input Z,
   output A,
   output B,
   output C,
   output D,
   output E,
   output F
);
assign A = X & Y;
assign B = X & (~Y || Z);
assign C = Z & (X ^ Y);
assign D = Y & ~Z;
assign E = 1'b0;
assign F = Z;
endmodule
module square_tb;
reg tb_X;
reg tb_Y;
reg tb_Z;
wire tb_A;
wire tb_B;
wire tb_C;
```

```
wire tb_D;
wire tb_E;
wire tb_F;
square dut(
.X(tb_X),
.Y(tb_Y),
.Z(tb_Z),
.A(tb_A),
.B(tb_B),
.C(tb_C),
.D(tb_D),
.E(tb_E),
.F(tb_F)
);
initial begin
tb_X = 1'b0; tb_Y = 1'b0; tb_Z = 1'b0;  #10;
tb_X = 1'b0; tb_Y = 1'b0; tb_Z = 1'b1;  #10;
tb_X = 1'b0; tb_Y = 1'b1; tb_Z = 1'b0;  #10;
tb_X = 1'b0; tb_Y = 1'b1; tb_Z = 1'b1;  #10;
tb_X = 1'b1; tb_Y = 1'b0; tb_Z = 1'b0;  #10;
tb_X = 1'b1; tb_Y = 1'b0; tb_Z = 1'b1;  #10;
tb_X = 1'b1; tb_Y = 1'b1; tb_Z = 1'b0;  #10;
tb_X = 1'b1; tb_Y = 1'b1; tb_Z = 1'b1;  #10;
$finish ;
end
initial
begin
$display("SQUARE OF A GIVEN 3-bit NUMBER ");
```

```
$display("--------------------------------------------");
$display("X  Y  Z    ||    A  B  C  D  E  F  ");
$display("--------------------------------------------");
$monitor("%b  %b  %b    ||    %b  %b  %b  %b  %b  %b   ", tb_X, tb_Y, tb_Z, tb_A,
tb_B, tb_C, tb_D, tb_E, tb_F);
end
endmodule
```

OUTPUT :

```
SQUARE OF A GIVEN 3-bit NUMBER
--------------------------------------------
X   Y   Z       ||      A   B   C   D   E   F
--------------------------------------------
0   0   0       ||      0   0   0   0   0   0
0   0   1       ||      0   0   0   0   0   1
0   1   0       ||      0   0   0   1   0   0
0   1   1       ||      0   0   1   0   0   1
1   0   0       ||      0   1   0   0   0   0
1   0   1       ||      0   1   1   0   0   1
1   1   0       ||      1   0   0   1   0   0
1   1   1       ||      1   1   0   0   0   1
```

AIM: Design a 4 bit parity generator and checker.

CODE :

```
module parity(
   input A,
   input B,
   input C,
   input D,
   output P,
   output checker
);

assign P = ~(A ^ B ^ C ^ D);
assign checker = ~(A ^ B ^ C ^ D ^ P);
endmodule

module parity_tb;
reg tb_A;
reg tb_B;
reg tb_C;
reg tb_D;
wire tb_P;
wire tb_checker;

parity dut(
```

```
.A(tb_A),
.B(tb_B),
.C(tb_C),
.D(tb_D),
.P(tb_P),
.checker(tb_checker)
);

initial begin
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b0; tb_D = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b0; tb_D = 1'b1; #10;
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b1; tb_D = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b0; tb_C = 1'b1; tb_D = 1'b1; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b0; tb_D = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b0; tb_D = 1'b1; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b1; tb_D = 1'b0; #10;
tb_A = 1'b0; tb_B = 1'b1; tb_C = 1'b1; tb_D = 1'b1; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b0; tb_D = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b0; tb_D = 1'b1; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b1; tb_D = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b0; tb_C = 1'b1; tb_D = 1'b1; #10;
tb_A = 1'b1; tb_B = 1'b1; tb_C = 1'b0; tb_D = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b1; tb_C = 1'b0; tb_D = 1'b1; #10;
tb_A = 1'b1; tb_B = 1'b1; tb_C = 1'b1; tb_D = 1'b0; #10;
tb_A = 1'b1; tb_B = 1'b1; tb_C = 1'b1; tb_D = 1'b1; #10;

$finish ;
end
```

```
initial
begin
$display("Parity Generator & Checker ");
$display("----------------------------------------");
$display("A   B   C   D    ||   Parity   Checker   ");
$display("----------------------------------------");
$monitor("%b   %b   %b   %b    ||    %b      %b    ", tb_A, tb_B, tb_C, tb_D, tb_P,
tb_checker);
end
endmodule
```

OUTPUT :

```
Parity Generator & Checker
----------------------------------------
A   B   C   D     ||    Parity   Checker
----------------------------------------
0   0   0   0     ||      1         0
0   0   0   1     ||      0         0
0   0   1   0     ||      0         0
0   0   1   1     ||      1         0
0   1   0   0     ||      0         0
0   1   0   1     ||      1         0
0   1   1   0     ||      1         0
0   1   1   1     ||      0         0
1   0   0   0     ||      0         0
1   0   0   1     ||      1         0
1   0   1   0     ||      1         0
1   0   1   1     ||      0         0
1   1   0   0     ||      1         0
1   1   0   1     ||      0         0
1   1   1   0     ||      0         0
1   1   1   1     ||      1         0
```